Unit I- Database Management and Web Development basics

Week 2:

Session 6 – SQL (Triggers)

1.Create a simple trigger BEFORE INSERT or update or delete trigger using the CREATE TRIGGER statement for driver schema assuming the otp for entry.

STEP 1: CREATE

```
mysql> CREATE DATABASE driver_schema;
Query OK, 1 row affected (0.03 sec)

mysql> USE driver_schema;
Database changed
mysql> CREATE TABLE driver (
    ->     driver_id INT PRIMARY KEY,
    ->     driver_name VARCHAR(50),
    ->     otp INT
    -> );
Query OK, 0 rows affected (0.05 sec)
```

STEP 2: BEFORE INSERT Trigger

```
mysql> DELIMITER $$
mysql> CREATE TRIGGER before_driver_insert
    -> BEFORE INSERT ON driver
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.otp IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'OTP is required for driver entry';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
mysql> INSERT INTO driver VALUES (1, 'Ravi', NULL);
ERROR 1644 (45000): OTP is required for driver entry
mysql> INSERT INTO driver VALUES (1, 'Ravi', 1234);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM driver;
+-----------+-------------+------+
| driver_id | driver_name | otp  |
+-----------+-------------+------+
|         1 | Ravi        | 1234 |
+-----------+-------------+------+
1 row in set (0.00 sec)
```

STEP 3: BEFORE UPDATE Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_driver_update
    -> BEFORE UPDATE ON driver
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.otp IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'OTP is required to update driver details';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE driver
    -> SET driver_name = 'Ravi Kumar', otp = NULL
    -> WHERE driver_id = 1;
ERROR 1644 (45000): OTP is required to update driver details
mysql> UPDATE driver
    -> SET driver_name = 'Ravi Kumar', otp = 5678
    -> WHERE driver_id = 1;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT * FROM driver;
+-----------+-------------+------+
| driver_id | driver_name | otp  |
+-----------+-------------+------+
|         1 | Ravi Kumar  | 5678 |
+-----------+-------------+------+
1 row in set (0.00 sec)
```

STEP 4: BEFORE DELETE Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_driver_delete
    -> BEFORE DELETE ON driver
    -> FOR EACH ROW
    -> BEGIN
    ->     IF OLD.otp IS NULL THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'OTP is required to delete driver record';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> DELETE FROM driver
    -> WHERE driver_id = 1 AND otp IS NULL;
Query OK, 0 rows affected (0.01 sec)

mysql> DELETE FROM driver
    -> WHERE driver_id = 1;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM driver;
Empty set (0.00 sec)
```

2. Create a simple trigger AFTER INSERT When a customer adds a dish to their order, the system must automatically calculate the price and add it to the order's running total.

STEP 1: Create table – dish ,orders ,order_items

```
mysql> CREATE DATABASE food_order;
Query OK, 1 row affected (0.01 sec)

mysql> USE food_order;
Database changed
mysql> CREATE TABLE dish (
    ->     dish_id INT PRIMARY KEY,
    ->     dish_name VARCHAR(50),
    ->     price DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE orders (
    ->     order_id INT PRIMARY KEY,
    ->     total_amount DECIMAL(10,2) DEFAULT 0
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE order_items (
    ->     order_item_id INT PRIMARY KEY,
    ->     order_id INT,
    ->     dish_id INT,
    ->     quantity INT
    -> );
Query OK, 0 rows affected (0.03 sec)
```

STEP 2: AFTER INSERT Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER after_insert_order_items
    -> AFTER INSERT ON order_items
    -> FOR EACH ROW
    -> BEGIN
    ->     DECLARE dish_price DECIMAL(10,2);
    ->
    ->     -- Get price of the dish
    ->     SELECT price INTO dish_price
    ->     FROM dish
    ->     WHERE dish_id = NEW.dish_id;
    ->
    ->     -- Add price × quantity to order total
    ->     UPDATE orders
    ->     SET total_amount = total_amount + (dish_price * NEW.quantity)
    ->     WHERE order_id = NEW.order_id;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO order_items VALUES (1, 101, 1, 2);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM orders;
Empty set (0.00 sec)

mysql> INSERT INTO orders VALUES (101, 0);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO dish VALUES (1, 'Pizza', 250);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM orders;
+----------+--------------+
| order_id | total_amount |
+----------+--------------+
|      101 |         0.00 |
+----------+--------------+
1 row in set (0.00 sec)
```

3. Automatically set created_at timestamp

STEP 1: Create Table

```
mysql> USE mysql;
Database changed
mysql> CREATE TABLE users (
    ->     id INT PRIMARY KEY,
    ->     name VARCHAR(50),
    ->     email VARCHAR(50),
    ->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.02 sec)
```

STEP 2: Insert

```
mysql> INSERT INTO users (id, name, email)
    -> VALUES (1, 'Amit', 'amit@example.com');
Query OK, 1 row affected (0.01 sec)
```

STEP 3: Output

```
mysql> SELECT * FROM users;
+----+------+------------------+---------------------+
| id | name | email            | created_at          |
+----+------+------------------+---------------------+
|  1 | Amit | amit@example.com | 2026-02-02 15:29:16 |
+----+------+------------------+---------------------+
1 row in set (0.00 sec)
```

4. Prevent negative salary entry by Stop inserting or updating an employee record if salary < 0.

STEP 1: Create Employee Table

```
mysql> CREATE DATABASE company;
Query OK, 1 row affected (0.01 sec)

mysql> USE company;
Database changed
mysql> CREATE TABLE employee (
    ->     emp_id INT PRIMARY KEY,
    ->     emp_name VARCHAR(50),
    ->     salary DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

STEP 2: Create BEFORE INSERT Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_employee_insert
    -> BEFORE INSERT ON employee
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.salary < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Salary cannot be negative';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.01 sec)
```

STEP 3: Create BEFORE UPDATE Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER before_employee_update
    -> BEFORE UPDATE ON employee
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.salary < 0 THEN
    ->         SIGNAL SQLSTATE '45000'
    ->         SET MESSAGE_TEXT = 'Salary cannot be negative';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.02 sec)
```

STEP 4: Test the Trigger

Insert

```
mysql> INSERT INTO employee VALUES (1, 'Ravi', -5000);
ERROR 1644 (45000): Salary cannot be negative
mysql> INSERT INTO employee VALUES (1, 'Ravi', 5000);
Query OK, 1 row affected (0.01 sec)
```

Update

```
mysql> UPDATE employee
    -> SET salary = -2000
    -> WHERE emp_id = 1;
ERROR 1644 (45000): Salary cannot be negative
```

5. Maintain audit log table: Whenever an employee record is updated, store old values in an employee_audit table.

STEP 1: Create Employee Table and Employee Audit Table

```
mysql> CREATE DATABASE employee_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE employee_db;
Database changed
mysql> CREATE TABLE employee_details (
    ->     emp_id INT PRIMARY KEY,
    ->     emp_name VARCHAR(50),
    ->     salary DECIMAL(10,2),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE employee_audit (
    ->     audit_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     emp_id INT,
    ->     old_name VARCHAR(50),
    ->     old_salary DECIMAL(10,2),
    ->     old_department VARCHAR(50),
    ->     updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    -> );
Query OK, 0 rows affected (0.02 sec)
```

STEP 2: Create AFTER UPDATE Trigger

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER after_employee_update
    -> AFTER UPDATE ON employee_details
    -> FOR EACH ROW
    -> BEGIN
    ->     INSERT INTO employee_audit (
    ->         emp_id,
    ->         old_name,
    ->         old_salary,
    ->         old_department
    ->     )
    ->     VALUES (
    ->         OLD.emp_id,
    ->         OLD.emp_name,
    ->         OLD.salary,
    ->         OLD.department
    ->     );
    -> END $$
Query OK, 0 rows affected (0.01 sec)
```

STEP 3: Test the Trigger

Insert employee

```
mysql> INSERT INTO employee_details
    -> VALUES (1, 'Ravi', 5000, 'IT');
Query OK, 1 row affected (0.01 sec)
```

Update employee

```
mysql> UPDATE employee_details
    -> SET salary = 7000, department = 'HR'
    -> WHERE emp_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

STEP 4: Output

```
mysql> SELECT * FROM employee_audit;
+----------+--------+----------+------------+----------------+---------------------+
| audit_id | emp_id | old_name | old_salary | old_department | updated_at          |
+----------+--------+----------+------------+----------------+---------------------+
|        1 |      1 | Ravi     |    5000.00 | IT             | 2026-02-02 17:55:05 |
+----------+--------+----------+------------+----------------+---------------------+
1 row in set (0.00 sec)
```

Session 7- SQL (Working with Date)

1.Find number of days between start_date and end_date.

```
mysql> SELECT id, name, DATEDIFF(end_date, start_date) AS total_days
    -> FROM records;
+----+------+------------+
| id | name | total_days |
+----+------+------------+
|  1 | A    |          9 |
|  2 | B    |         15 |
|  3 | C    |         21 |
|  4 | D    |         23 |
+----+------+------------+
4 rows in set (0.01 sec)
```

2. Find the expiry date after 30 days.

```
mysql> SELECT id, name, DATE_ADD(start_date, INTERVAL 30 DAY) AS expiry_date
    -> FROM records;
+----+------+-------------+
| id | name | expiry_date |
+----+------+-------------+
|  1 | A    | 2026-03-03  |
|  2 | B    | 2026-03-07  |
|  3 | C    | 2026-03-09  |
|  4 | D    | 2026-03-12  |
+----+------+-------------+
4 rows in set (0.00 sec)
```

3. Get Date before seven days from today.

```
mysql> SELECT DATE_SUB(CURDATE(), INTERVAL 7 DAY) AS before_7_days;
+---------------+
| before_7_days |
+---------------+
| 2026-01-27    |
+---------------+
1 row in set (0.00 sec)
```

4. Get the records created on weekend dates.

```
mysql> SELECT *
    -> FROM records
    -> WHERE DAYOFWEEK(created_date) IN (1, 7);
+----+------+------------+------------+--------------+
| id | name | start_date | end_date   | created_date |
+----+------+------------+------------+--------------+
|  1 | A    | 2026-02-01 | 2026-02-10 | 2026-02-01   |
|  2 | B    | 2026-02-05 | 2026-02-20 | 2026-02-08   |
|  4 | D    | 2026-02-10 | 2026-03-05 | 2026-02-15   |
+----+------+------------+------------+--------------+
3 rows in set (0.01 sec)
```

5. Read input date as string then convert to date format 'DD-MM-YYYY' then display the month name from the date given.

```
mysql> SELECT
    -> MONTHNAME(STR_TO_DATE('15-02-2026', '%d-%m-%Y')) AS month_name;
+------------+
| month_name |
+------------+
| February   |
+------------+
1 row in set (0.01 sec)
```

Session 8 – SQL (Indexes and Views)

1.Improve search performance on email column using CREATE INDEX.

```
mysql> CREATE INDEX idx_email ON users(email);
Query OK, 0 rows affected (0.05 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM users WHERE email = 'jeeva@gmail.com';
+----+----------+----------------+--------------+--------+---------------------+
| id | username | email          | aadhar       | status | created_at          |
+----+----------+----------------+--------------+--------+---------------------+
|  1 | jeeva    | jeeva@gmail.com | 123456789012 | active | 2026-02-01 10:00:00 |
+----+----------+----------------+--------------+--------+---------------------+
1 row in set (0.01 sec)
```

2. Create UNIQUE INDEX to ensure no duplicate values in username.

```
mysql> CREATE UNIQUE INDEX idx_unique_username ON users(username);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3. Find and remove the unused index from the table using DROP INDEX.

```
mysql> SHOW INDEX FROM users;
+-------+------------+----------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+--------------+---------+------------+
| Table | Non_unique | Key_name             | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-------+------------+----------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+--------------+---------+------------+
| users |          0 | PRIMARY              |            1 | id          | A         |           3 |     NULL |   NULL |      | BTREE      |         |              | YES     | NULL       |
| users |          0 | idx_unique_username  |            1 | username    | A         |           3 |     NULL |   NULL | YES  | BTREE      |         |              | YES     | NULL       |
| users |          1 | idx_email            |            1 | email       | A         |           3 |     NULL |   NULL | YES  | BTREE      |         |              | YES     | NULL       |
+-------+------------+----------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+--------------+---------+------------+
3 rows in set (0.04 sec)

mysql> DROP INDEX idx_email ON users;
Query OK, 0 rows affected (0.02 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4. A SELECT query using WHERE aadhar= ? is slow. Fix it using index.

```
mysql> SELECT * FROM users WHERE aadhar = '123456789012';
+----+----------+-----------------+--------------+--------+---------------------+
| id | username | email           | aadhar       | status | created_at          |
+----+----------+-----------------+--------------+--------+---------------------+
|  1 | jeeva    | jeeva@gmail.com | 123456789012 | active | 2026-02-01 10:00:00 |
+----+----------+-----------------+--------------+--------+---------------------+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_aadhar ON users(aadhar);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

5. Create a composite index: (status, created_at) to query filters by status and sorts by created_at.

```
mysql> CREATE INDEX idx_status_created
    -> ON users(status, created_at);
Query OK, 0 rows affected (0.03 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT *
    -> FROM users
    -> WHERE status = 'active'
    -> ORDER BY created_at;
+----+----------+-----------------+--------------+--------+---------------------+
| id | username | email           | aadhar       | status | created_at          |
+----+----------+-----------------+--------------+--------+---------------------+
|  1 | jeeva    | jeeva@gmail.com | 123456789012 | active | 2026-02-01 10:00:00 |
|  3 | kavi     | kavi@gmail.com  | 456789123456 | active | 2026-02-03 09:30:00 |
+----+----------+-----------------+--------------+--------+---------------------+
2 rows in set (0.00 sec)
```

Session 9 – SQL (SQL Built-in Functions)

1.Extract the letters before @ from the username and append 123 at the end to create a default password.

```
mysql> SELECT
    ->     username,
    ->     CONCAT(SUBSTRING_INDEX(username, '@', 1), '123') AS default_password
    -> FROM user_details;
+------------------+------------------+
| username         | default_password |
+------------------+------------------+
| arun@gmail.com   | arun123          |
| meena@yahoo.com  | meena123         |
| kavi@outlook.com | kavi123          |
+------------------+------------------+
3 rows in set (0.01 sec)
```

2. Correct the invalid contact numbers which contains spaces, dashes or any other symbols. Keep only numbers to make the contact valid.

```
mysql> SELECT
    ->     contact,
    ->     REGEXP_REPLACE(contact, '[^0-9]', '') AS valid_contact
    -> FROM user_details;
+---------------+---------------+
| contact       | valid_contact |
+---------------+---------------+
| 987-654-3210  | 9876543210    |
| 98 76 54 32   | 98765432      |
| (987)654-1111 | 9876541111    |
+---------------+---------------+
3 rows in set (0.01 sec)
```

3. Normalise the employee names given in mixed case using UPPER(), LOWER(), SUBSTRING(), CONCAT() functions.

```
mysql> SELECT
    ->     emp_name,
    ->     CONCAT(
    ->         UPPER(SUBSTRING(emp_name, 1, 1)),
    ->         LOWER(SUBSTRING(emp_name, 2))
    ->     ) AS normalised_name
    -> FROM employees;
+-------------+-----------------+
| emp_name    | normalised_name |
+-------------+-----------------+
| raVi kuMar  | Ravi kumar      |
| MEENA devi  | Meena devi      |
| aRun sharma | Arun sharma     |
| kAVI        | Kavi            |
+-------------+-----------------+
4 rows in set (0.01 sec)
```

4. Handle multiple fallback columns in the use table with the priority of contact details as Mobile→Phone→email. Hint: Use COALESCE.

```
mysql> SELECT
    ->     username,
    ->     COALESCE(mobile, phone, email) AS preferred_contact
    -> FROM user_details;
+------------------+-------------------+
| username         | preferred_contact |
+------------------+-------------------+
| arun@gmail.com   | 044 234567        |
| meena@yahoo.com  | 9876543211        |
| kavi@outlook.com | kavi@outlook.com  |
+------------------+-------------------+
3 rows in set (0.01 sec)
```

5. Find average experience per department using the functions AVG(), TIMESTAMPDIFF(), GROUP BY.

```
mysql> SELECT
    ->     department,
    ->     AVG(TIMESTAMPDIFF(YEAR, join_date, CURDATE())) AS avg_experience
    -> FROM employees
    -> GROUP BY department;
+------------+----------------+
| department | avg_experience |
+------------+----------------+
| IT         |         7.0000 |
| HR         |         7.0000 |
+------------+----------------+
2 rows in set (0.01 sec)
```

Session 10 – SQL (User-Defined Functions)

1.Create a function to calculate net salary after 10% tax deduction.

```
mysql> USE date_functions_demo;
Database changed
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION calc_net_salary(gross_salary DECIMAL(10,2))
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->     RETURN gross_salary - (gross_salary * 0.10);
    -> END $$
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT calc_net_salary(50000) AS net_salary;
+------------+
| net_salary |
+------------+
|   45000.00 |
+------------+
1 row in set (0.01 sec)
```

2. Create a function to check activity of the user. Return Active if last login within 30 days, else Inactive.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION check_user_activity(last_login DATE)
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> BEGIN
    ->     IF DATEDIFF(CURDATE(), last_login) <= 30 THEN
    ->         RETURN 'Active';
    ->     ELSE
    ->         RETURN 'Inactive';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT check_user_activity('2026-01-15') AS user_status;
+--------------+
| user_status |
+--------------+
| Active      |
+--------------+
1 row in set (0.01 sec)
```

3. Create a function to perform dynamic tax slab calculation based on the following categories:

Salary	Tax %
≤ 3,00,000	0
3,00,001-6,00,000	10
6,00,001– 10,00,000	20
> 10,00,000	30

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION calc_tax(salary DECIMAL(10,2))
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE tax DECIMAL(10,2);
    ->
    ->     IF salary <= 300000 THEN
    ->         SET tax = 0;
    ->     ELSEIF salary <= 600000 THEN
    ->         SET tax = salary * 0.10;
    ->     ELSEIF salary <= 1000000 THEN
    ->         SET tax = salary * 0.20;
    ->     ELSE
    ->         SET tax = salary * 0.30;
    ->     END IF;
    ->
    ->     RETURN tax;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT calc_tax(750000) AS tax_amount;
+------------+
| tax_amount |
+------------+
|  150000.00 |
+------------+
1 row in set (0.01 sec)
```

4. Create a function to categorize the employees based on the years of experience: Years Category < 2 Fresher 2–5 Junior 6–10 Mid > 10 Senior

Years	Category
<2	Fresher
2-5	Junior
6-10	Mid
>10	Senior

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION emp_category(exp_years INT)
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> BEGIN
    ->     IF exp_years < 2 THEN
    ->         RETURN 'Fresher';
    ->     ELSEIF exp_years BETWEEN 2 AND 5 THEN
    ->         RETURN 'Junior';
    ->     ELSEIF exp_years BETWEEN 6 AND 10 THEN
    ->         RETURN 'Mid';
    ->     ELSE
    ->         RETURN 'Senior';
    ->     END IF;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT emp_category(7) AS category;
+----------+
| category |
+----------+
| Mid      |
+----------+
1 row in set (0.00 sec)
```

5. Create a late fee calculator using function where late fee of ₹50/day is applicable after due date limited to as maximum amount of ₹1000.

```
mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION calc_late_fee(due_date DATE, payment_date DATE)
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE late_days INT;
    ->     DECLARE fee INT;
    ->
    ->     SET late_days = DATEDIFF(payment_date, due_date);
    ->
    ->     IF late_days <= 0 THEN
    ->         SET fee = 0;
    ->     ELSE
    ->         SET fee = late_days * 50;
    ->         IF fee > 1000 THEN
    ->             SET fee = 1000;
    ->         END IF;
    ->     END IF;
    ->
    ->     RETURN fee;
    -> END $$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT calc_late_fee('2026-01-01', '2026-01-30') AS late_fee;
+----------+
| late_fee |
+----------+
|     1000 |
+----------+
1 row in set (0.00 sec)
```